

COMP2040 – Operating Systems

Lab 01: C Foundations, OS Services and System Calls

Overview

This lab is deliberately small. You are not asked to build a large program; the point is to see, in concrete terms, how a line of C source code ends up as a request to the operating system.

After this lab you should be able to

- compile and run a C program from the terminal;
- explain what a header file is for, and build a program from several source files;
- do basic file I/O with `open()`, `write()`, `read()` and `close()`;
- use `stat()` to look up the size and permissions of a file;
- relate what you observe to the Week 2 material on OS services and system calls.

Setup

You need a terminal (Linux, macOS Terminal, or Ubuntu under WSL on Windows) with `gcc` available. The recommended editor is Sublime Text, but any editor or IDE is fine as long as you can compile and run your programs from the terminal. On Windows, work inside WSL; the POSIX calls used below are not available in a native Windows toolchain. The separate *C Environment Setup* guide covers installation on each platform.

Check that the compiler works before you start:

```
| gcc --version
```

If this fails, ask a TA before going further.

The starter `.c` and `.h` files come with this handout. Keep the given file names, otherwise the commands below will not work as written.

Submission

Upload two files to Canvas before the deadline shown there:

1. `StudentID_Lab01.pdf` – a short report;
2. `StudentID_Lab01.zip` – all your completed `.c` and `.h` files.

The report is your evidence of what you ran and what you understood. For each task, give the command(s) you used, the relevant terminal output (copied as text rather than screenshots where possible), and short answers to the questions. There is no need to paste your source code, since it is submitted separately. Keep answers specific to what you actually observed; two to four sentences is usually enough.

1 Edit, compile, run

Open `task1_hello.c` and read it before compiling. Compile with warnings enabled and run the result:

```
gcc -Wall -Wextra task1_hello.c -o task1_hello
./task1_hello
```

Change the printed message so that it includes your first name, then compile and run again.

Include in your report.

- The compile command and the final output.
- Which file did `gcc` produce after a successful compilation?
- `printf()` is a C library function, not a system call. Explain the difference in one or two sentences, using the Week 2 material.

Hint: compare `man 3 printf` (the library function) with `man 2 write` (the system call that terminal output normally goes through).

2 Header files and multiple source files

The `task2` directory contains three files:

- `math_operations.h` – function declarations;
- `math_operations.c` – function definitions;
- `main.c` – a program that calls those functions.

`multiply()` and `divide_numbers()` are unfinished; complete both. Then compile the two source files together and run:

```
gcc -Wall -Wextra main.c math_operations.c -o math_demo
./math_demo
```

The program should print the correct sum, difference, product and quotient for the values in `main.c`. Division by zero must be handled without crashing.

Include in your report.

- The compile command and the program output.
- In this example, what goes in the `.h` file and what goes in the `.c` file?
- Why does `main.c` include `math_operations.h`?

3 File I/O with system calls

Open `task3_file_io.c` and complete the `TODD` sections so that the program does the following, in order:

open (write) → write → close → open (read) → read → close

Specifically, it must

1. create (or overwrite) a file called `lab01.txt`;
2. write the given message to it with `write()`;
3. close the file;
4. open the same file for reading;
5. read the contents back with `read()`;
6. print the number of bytes read and the text itself;
7. close the file before exiting.

If `open()`, `read()`, `write()` or `close()` fails, report it with `perror()` or an equally clear message. You may use `printf()` to display results, but the file operations themselves must use the four system calls above.

```
gcc -Wall -Wextra task3_file_io.c -o task3_file_io
./task3_file_io
cat lab01.txt
```

Include in your report.

- (a) Your commands and the relevant output.
- (b) What does a successful `open()` return, and why is that value passed to `read()`, `write()` and `close()`?
- (c) Compare the number of bytes you asked for with the value returned by `read()` or `write()`. What does the return value tell you?

4 File size with `stat()`

`task4_file_size.c` already takes a file name as a command-line argument. Complete the **TODO** sections so that it calls `stat()` and prints the file size in bytes from `st_size`.

```
gcc -Wall -Wextra task4_file_size.c -o task4_file_size
./task4_file_size lab01.txt
```

Now open `lab01.txt` in your editor, add one more line of text, save, and run the program again.

Include in your report.

- (a) The file size before and after the edit.
- (b) Which member of `struct stat` holds the file size?
- (c) Explain why the size changed by exactly the amount you saw.

5 File permissions with `stat()`

Complete `task5_permissions.c` so that it uses `stat()` and `st_mode` to print the nine permission characters for owner, group and others, e.g. `rw-r-r-`. The masks you need (`S_IRUSR`, `S_IWUSR`, `S_IXUSR`, ...) are listed in the starter file.

```
gcc -Wall -Wextra task5_permissions.c -o task5_permissions
./task5_permissions lab01.txt
ls -l lab01.txt
```

Check that the two outputs agree. Then change one owner permission, for example

```
chmod u+x lab01.txt
```

and run both checks again.

Include in your report.

- (a) The permission string before and after `chmod`.
- (b) Which bit (mask) of `st_mode` changed?
- (c) In Week 2 terms, how do file permissions relate to the OS service of protection and security?